# Event-Driven Hierarchical Rule Composition and Saturation

Coarse and Smooth Layers, Dissipative and Infi Rule Compositions

Technical report 23 September 2026
Implementation baseline: `df8b122` Provenance/runtime revision v0.2.5

**Abstract**

Equality saturation shares concrete expressions, but repeated derivations with different bindings can still generate large graphs and dense execution histories. We present a hierarchy that treats rule applications as evidence for reusable reasoning units. Committed events form a versioned dependency graph. Relative to a selected context, applications are classified as coarse or smooth and organized into coarse layers, smooth layers, and CS units. Sequential CSCS composition preserves dependencies; guarded CCSS regrouping combines the coarse and smooth parts without discarding interface obligations. An explicit *ripen* operation computes local closure under staged external inputs, while exact state comparison preserves types, equality, visibility, and the relevant rule environment. We then define parameterized fractal rule compositions through stable binding and effect transfers, distinguishing well-founded dissipative processes from families with arbitrarily long finite prefixes. These abstractions suggest finite scheduling, endpoint reduction, and demand-driven symbolic feedback to ripen. The current Rust/native prototype separates full tier0 provenance from local ripen effect observation, reuses typed queries, and parallelizes isolated work. A three-round math workload yields 79 saturated jobs, including shared aliases, and six unfinished jobs; three graph-free process times are 1.33, 1.06, and 1.07 seconds. These are local-interface observations, not tier0 compression or acceleration. We propose compiling optional local histories into origin templates indexed by caller bindings and fact slots, while distinguishing reusable derivation evidence from the actual producer of a committed fact. That feedback, general recurrence induction, and live tier0 replacement are not implemented.

**Keywords:** equality saturation; rule composition; local closure; relative binding; event-driven analysis; recursive summaries.

## 1 Introduction

An e-graph shares e-nodes with identical operators and canonical children, and records equivalence between expressions through union. This eliminates concrete structural duplication. It does not automatically share a derivation such as expansion, commutation, reassociation, and folding across many different bindings. Even when the expression graph shares some nodes, the history of matches and the space of potential rule combinations may continue to grow.

Our objective is to discover reusable reasoning units that can eventually reduce repeated storage and matching. Three objects must remain distinct: a state, a process reaching that state, and the conditions enabling the process. Different processes may reach the same state, while the same rule sequence may behave differently under different bindings and external effects. Neither a list of rule names nor a single endpoint expression is therefore an adequate sharing key.

Egglog combines Datalog-style reasoning with equality saturation and provides the native execution substrate for our prototype [1]. We use *tier0* for ordinary execution, *tier1* for the analysis of applications

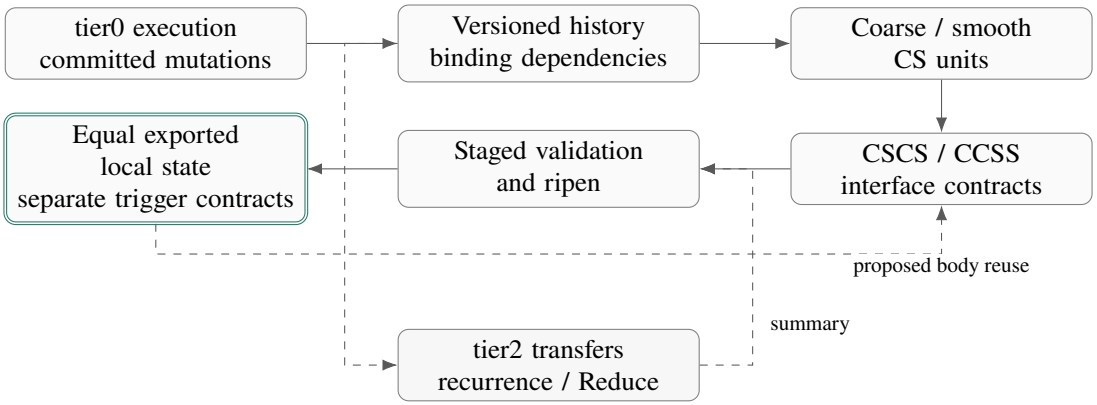



Figure 1: The hierarchy. Dashed connections mark proposed abstraction or feedback paths. The current registry constructs descriptors from recorded consumers; these feedback paths do not implement live tier0 replacement.

and their compositions, and *tier2* for binding transfers, recurrence evidence, and symbolic reduction. These are logical layers: the current tier1 is primarily a Rust data structure rather than a mandatory chain-shaped egglog encoding.

This report gives a scoped implementation account of the hierarchy and audits its evidence boundaries. The implemented contribution is narrower than the surrounding vocabulary: observed applications are grouped into candidate units, candidate compositions are replayed in isolated native cells, and exported local states can be compared and catalogued. The prototype does not replace tier0 execution, discover all useful combinations, prove arbitrary-depth recurrences, or perform runtime storage compression. The diagrams therefore distinguish observed history, replayed local state, and proposed summaries instead of presenting them as one execution pipeline.

### 1.1 How the mechanisms fit together

Read the finite pipeline in this order: recorded applications supply evidence; CSCS and CCSS propose candidate interfaces; native replay validates those interfaces; ripen computes a local fixed point; exact comparison groups equivalent bodies. These operations work on different objects. Historical promotion experiments fed validated member descriptors back into discovery. The current registry interns compositions from recorded consumers; it does not use the exported saturated body as an executable macro. The later sections develop parameterized families and summary contracts as an extension of this finite pipeline. Their definitions and examples remain useful even where the execution feedback is not yet integrated.

### 1.2 Reading the running examples

The examples use actual rewrite bodies from `egglog/tests/web-demo/math.egg`. We isolate small rule sets and explicit schedules so that the illustrated dependency is reproducible; this is not a claim that the complete optimizer reaches the same saturated body. In the running Add example, $a, b, c$ abbreviate the concrete eclasses containing `Var("a")`, `Var("b")`, and `Var("c")`. For commutation we alpha-rename its pattern variables to $x, y$, keeping these formal names distinct from the concrete leaf labels. For the native e-graph pictures we also supply a minimal, standalone program, `examples/math-mini-trace.egg`. It keeps only Var, Add, associativity and commutation. First inspect the input $a + (b + c)$, then the two orange nodes added by one associativity application, then the complete saturated graph. The topology is exported by egglog; the paper script only annotates it. This gives one concrete thread through the interface, composition and saturation definitions.

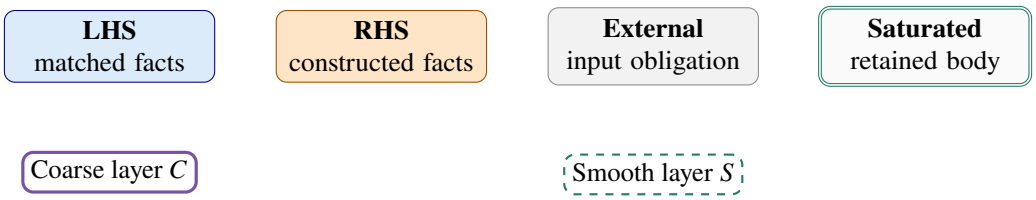



Figure 2: The v0.2.1 palette is retained. In rule diagrams blue/orange mark matched/constructed facts, and purple/teal outlines mark coarse/smooth layers. In native e-graph snapshots blue means present at entry and orange means added since entry; purple/teal dashed clusters identify child/root e-classes, not layers. Each caption specifies which kind of object and edge is being shown.

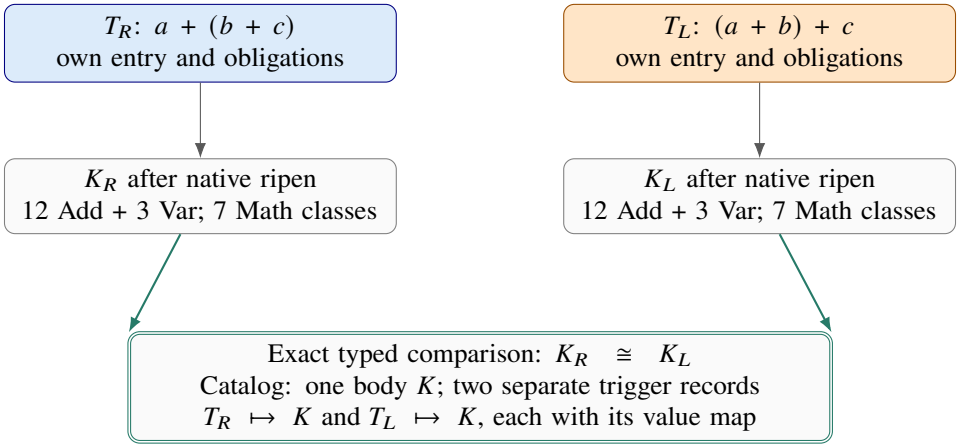



Figure 3: Two different entries can produce the same body. These are catalog and execution records, not e-class boxes. Native replays and exact comparison are checked by the `math_parenthesizations` regression. Equal counts alone are insufficient: the comparison preserves all rows, ports and visibility. The two entries remain distinct, and the live tier0 graph is not replaced.

## 2 State and Execution Evidence

### 2.1 Facts, equality, visibility, and updates

We distinguish the components of a runtime state as

$$G = (F, \equiv, V, \Theta, \delta). \tag{1}$$

Here $F$ contains constructor and relation facts, $\equiv$ is equality, $V$ records row visibility, $\Theta$ contains function tables and their update semantics, and $\delta$ denotes execution and incremental-maintenance state. The supplied rewrite theory is trusted; native replay is not an independent proof of every rule as a mathematical identity. A union is not an ordinary row insertion; subsumption is not simply the absence of a semantic fact; and a function update is not necessarily set union.

For a positive fragment that only adds facts and grows equality, a monotone closure model is useful. General programs require a stronger operational contract. A subsumed row can remain stored while ceasing to participate in matching. Comparing only its contents would lose this distinction.

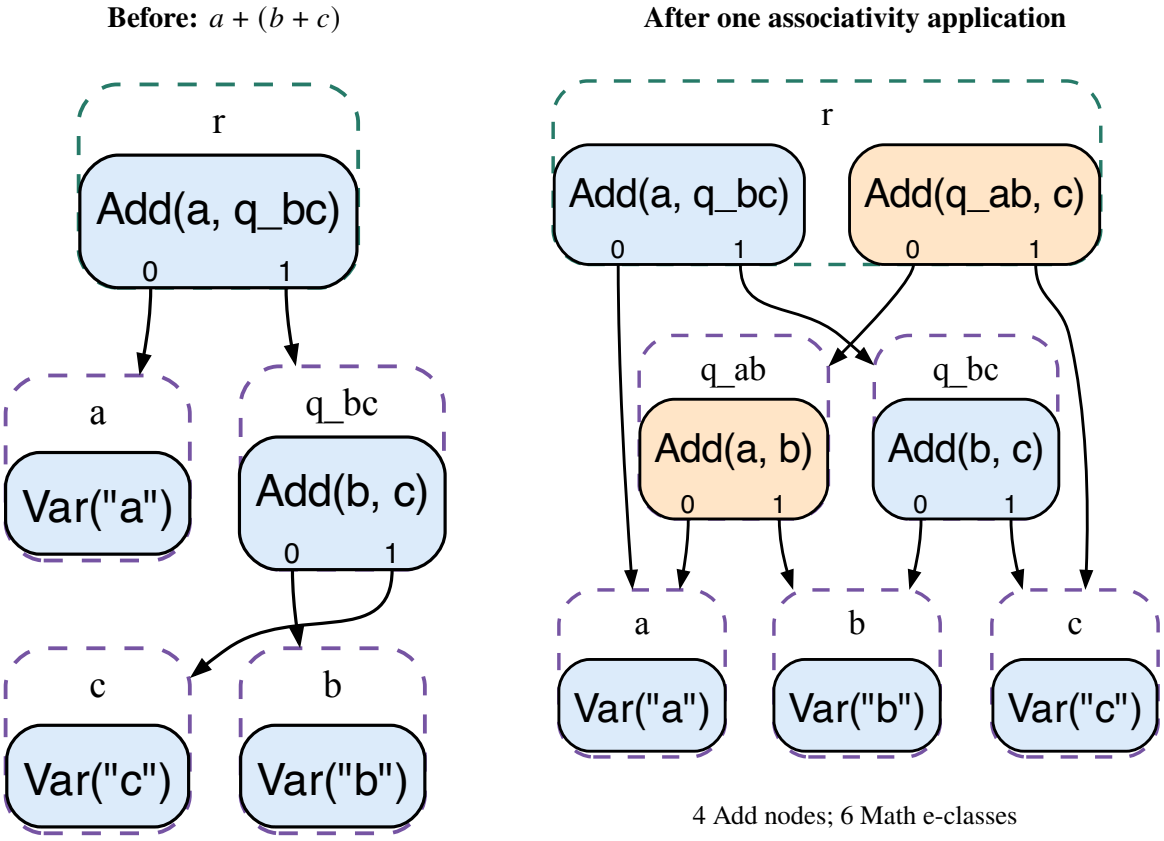



Figure 4: Read the change from left to right. Native egglog introduces Add$(a, b)$ and Add$(q_{ab}, c)$ (orange) and unions the latter with the existing root $r$. Blue nodes were already present. Dashed enclosures are native e-classes; numbered ports are argument positions, and arrows terminate at child classes. Both graphs come from executable prefixes of `math-mini-trace.egg`, through egglog's own DOT exporter. Var's String payloads are inlined into its label.

### 2.2 Committed applications rather than matches

An application record is modeled as

$$a_i = (R_i, \beta_i, Q_i, \Delta_i, U_i, W_i), \tag{2}$$

where $R_i$ is a rule, $\beta_i$ its binding, $Q_i$ the facts and equality evidence read, $\Delta_i$ the committed writes, $U_i$ the union information, and $W_i$ update or visibility information. A logical match, a surviving action lane, an inserted row, and a deduplicated write are different events. Our provenance must not silently identify them.

An edge from $a_i$ to $a_j$ records that $a_j$ used a then-valid fact version or equality witness supplied by $a_i$. Equal values can suggest interaction, but value equality alone does not establish this causal edge. The event graph is acyclic in time even when a rule consumes its own earlier output. Cycles in a quotient graph of recurring patterns instead denote recursion between pattern instances, not cycles in the original event history.

### 2.3 Relative binding as an interface map

A relative binding maps an input of one unit to an output, a carried input, or a new external port:

$$\rho : \mathrm{In}(B) \longrightarrow \mathrm{Out}(A) \cup \mathrm{In}(A) \cup E. \tag{3}$$

It also carries types, alias constraints, and equality evidence. Such a map may be a permutation, a projection through constructor arguments, a construction of new terms, or a reference to another application's result. The notation $\beta' = \beta + \Delta\beta$ is appropriate only for an actually additive coordinate system; it is not a general definition of relative binding.

Definitions and uses follow a dependency order. A later application may refer to an earlier binding, but a forward reference or a collision between names from different instances is invalid. Numbering variables by first appearance alone is not a complete canonicalization of arbitrary branching graphs.

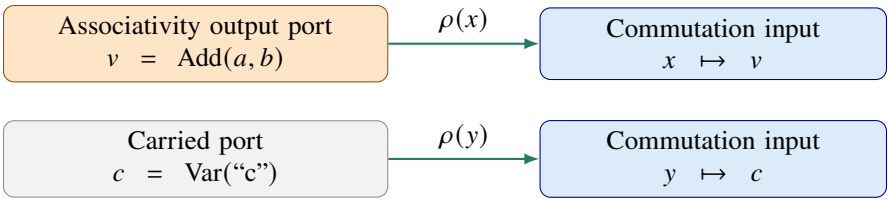



Figure 5: A concrete relative binding from the native Add trace: the next rule matches $\mathrm{Add}(v, c)$, so its formal variables become $x = v$ and $y = c$. This is a port map, not addition of numeric binding offsets. The producer-to-consumer row witness is recorded in the evidence file.

## 3 Coarse and Smooth Rule Compositions

Let $H$ be a selected local unit. Immediately before its $i$th stage, let $A_i(H)$ be the evidence supplied by already injected inputs and earlier internal actions. For the chosen recorded witness, define the new external obligation conceptually as

$$E_i(H) = Q_i \setminus A_i(H). \tag{4}$$

The new injection for stage $i$ is not yet included in $A_i(H)$. By convention, the initial triggering input is charged to the first coarse step. This is evidence subtraction, not subtraction of bare IDs: equality, versions, and visibility must be respected.

**Definition 1** (Context-relative application kind)**.** *A one-step composition is a* coarse rule composition *relative to $H$ if it requires a nonempty new external obligation $E_i(H)$. It is a* smooth rule composition *if all its premises are explained by the current local interface and prior internal results.*

The classification belongs to an application in a chosen combination, not to the rule permanently. A coarse application can become smooth when its suppliers are included in a larger unit. Classification also depends on the selected witness: a different internal proof may exist without having been discovered. Missing provenance is not proof that no such internal derivation exists.

For the running example, take Add associativity and commutation from `math.egg`. Starting from external $a + (b + c)$, associativity is coarse. Commutation of its produced row $(a + b) + c$, with $x = [a + b]$ and $y = c$, is smooth relative to that two-step unit. Viewed alone, the same commutation requires an external row and is coarse. Figure 6 gives a second concrete case: merely having the denominator binding does not provide its nonzero guard.

A nonempty-premise combination normally begins with a coarse injection. A rule with no premises is a logical exception. An engineering convention such as `Empty -> coarse rule composition` can provide a uniform entry representation, but does not prove that every rule needs external data.

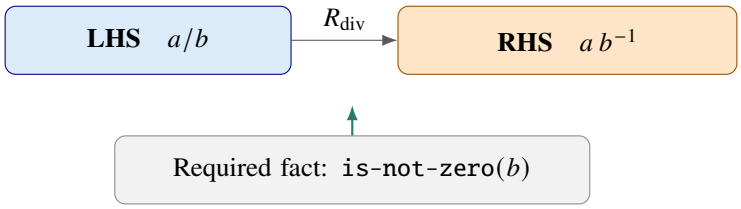



Figure 6: Why a binding is not enough. The guarded division rule is copied from `math.egg`: a native check confirms that it does not apply without `is-not-zero`($b$) and applies after that fact is supplied. This illustrates an external obligation; it does not independently establish that the supplied mathematical assertion is true.

## 4 Layers and CS Units

A *coarse layer* groups coarse applications with explicit binding anchors. A *smooth layer* contains applications whose relevant dependencies can be explained behind that interface. We write a CS unit as

$$U = (C, S, I, O, \Pi), \tag{5}$$

with coarse members $C$, smooth members $S$, external interface $I$, output ports $O$, and witness information $\Pi$.

Figure 7 makes the layer distinction concrete. The native trace contains an associativity write that is subsequently read by commutation with $x = [a + b]$ and $y = c$. A layer encloses applications; its outline must not be confused with the blue/orange roles of facts inside each application.

An intersection is a reason to inspect a pair, not automatically a reason to merge it. For example, merging transitively on the shared constant zero can destroy locality. A common smooth consumer, explicit port connections, and coverage checks provide stronger selection signals. The current discovery path does not truncate this work with the historical partner or candidate-count limits. Requiring a consumer to use outputs from each participating coarse member is a useful heuristic, not a semantic-equivalence theorem.

Units need not form a disjoint partition. A fact can be exposed by several interfaces, and a history can admit multiple useful decompositions. Sharing references and retaining alternatives avoids permanently committing to an early partition.

A *Use* is a concrete occurrence of an installed rule-combination template. It records a template ID, member events, and actual port bindings. This differs from a CS unit, which is defined by contextual obligations. The earlier pipeline admitted only Uses to ripen. The current pipeline registers witnessed CS interfaces directly; Use-based admission is historical terminology, not an additional required stage or a default alternative source of jobs.

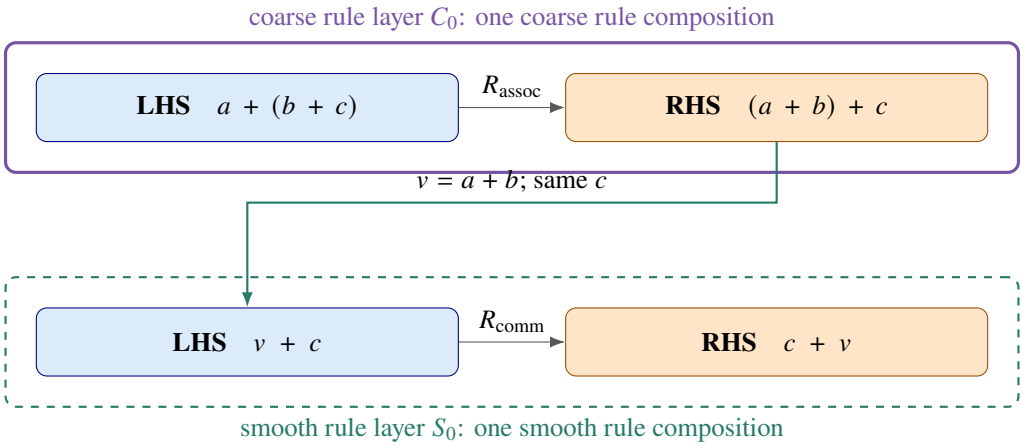



Figure 7: A concrete CS unit from `math.egg`. The first step requires the external entry $\text{Add}(a, \text{Add}(b, c))$; the second reads the associativity output and needs no new external fact. The repeated orange/blue expression is one reused node, shown in two occurrence roles, not duplicated storage. Labels $C_0$, $S_0$ describe this selected interface, not globally unique runtime layer IDs.

# 5 Two Candidate Composition Operators

## 5.1 Sequential CSCS composition

If $U_2$ consumes outputs of $U_1$, we represent their composition as

$$\text{Seq}(U_1, U_2, \rho). \tag{6}$$

The map $\rho$ separates inputs supplied by $U_1$ from inputs that still require external injection. It preserves the dependency direction rather than simply concatenating two event arrays. If both units contain applications depending on intermediate results of the other, the complete units cannot be placed in a simple sequential order. A finer decomposition or an unknown result is needed. Dependencies through events outside the proposed pair must also be checked.

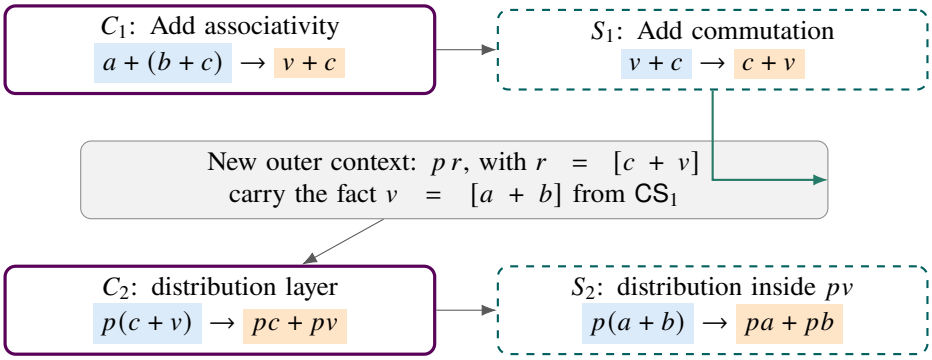



Figure 8: A native-checked CSCS example using associativity, commutation, and distribution from `math.egg`. Distribution may match several retained representatives; one is shown. The declared interface of $\text{CS}_2$ carries the $\text{Add}(a, b)$ fact, so its inner distribution step does not fetch it for free. The fresh outer multiplication context is supplied only after $\text{CS}_1$.

The multiplication context in Figure 8 is a genuinely new external obligation. The carried $\text{Add}(a, b)$ fact is also explicit: omitting it would incorrectly classify the later inner distribution as self-contained. The diagram shows an interface-level grouping of a native trace; a runtime heuristic may choose a finer layer cut.

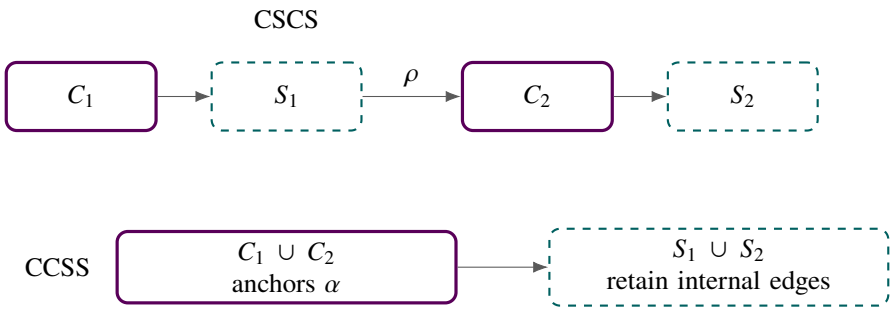



Figure 9: CSCS preserves a cross-unit dependency. CCSS regroups C and S only under its contract. Neither operation removes external-input obligations.

## 5.2 Guarded CCSS regrouping

The second operator regroups the coarse and smooth parts:

$$\mathsf{Par}(U_1, U_2, \alpha) = (C_1 \cup C_2,\ S_1 \cup S_2,\ I', O', \Pi'). \tag{7}$$

Here $\alpha$ gives typed anchors between the C parts. The symbol Par names regrouping, not a parallel execution schedule. Dependencies within the combined C or S DAG remain edges.

Our sufficient admission conditions are conservative: disjoint member events, an explicit coarse-port intersection, supported positive constructor/relation/equality operations, and no transitive dependency from any S member to any C member. Failure to obtain a complete dependency explanation is unknown, not evidence of independence.

**Proposition 1** (Conditional regrouping)**.** *Fix the external-input obligations and a complete dependency witness. If no C member depends on an S member, a topological order exists that places C before S. For supported positive monotone actions, changing between legal internal orders preserves the resulting deductive closure.*

*Proof.* All predecessors of C lie outside S, so the C subgraph can be topologically ordered before the S subgraph. Every selected premise remains available because internal producers precede consumers and the external obligations are fixed. Independent additions and unions do not retract earlier evidence. Recomputing closure under the same positive theory therefore preserves the consequences. □

This proposition does not move a late external fact earlier. The implementation validates the original staged sequence; its CCSS certificate is conditional on interface obligations, not a claim that the surrounding tier0 execution has been rescheduled. Function updates, subsume, and unsupported guards do not pass the positive-fragment regrouping check.

## 5.3 Anchors need not be an initial intersection

Suppose one unit supplies $P(a, b)$, another supplies $Q(c, d)$, and a recorded consumer also reads $Link(b, c)$. The consumer and the Link fact anchor a combination even if the original C parts shared no node. This is a more general interaction witness than a direct coarse intersection. It must not be forced into the particular CCSS admission test.

The registry stores explicit composition references and flattens their member references for staged validation. Earlier prototypes tested promotion of saturated descriptors. In the current path, composite registration follows observed consumer dependencies; it does not install a previously exported body as a new execution input or automatically find every legal regrouping.

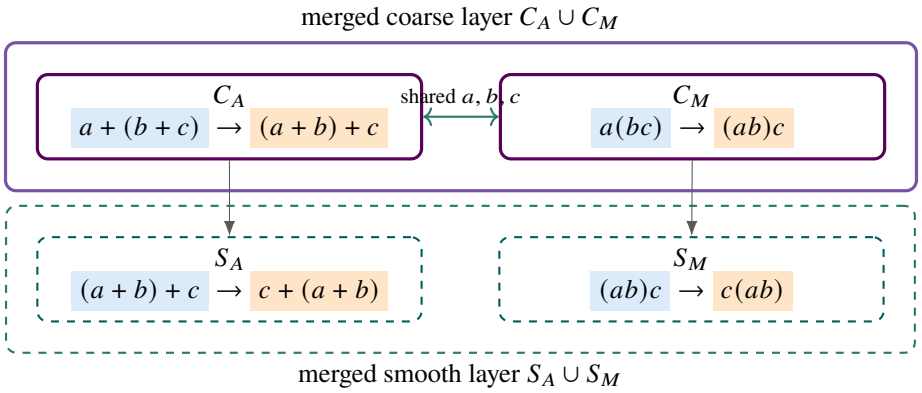



Figure 10: CCSS with real Add/Mul rules from `math.egg`. Both input trees are declared initially and share $a, b, c$; there is no S-to-C dependency. The restricted native fragment saturates to 12 Add and 12 Mul rows. Layer outlines indicate regrouping, not deletion of the four original compositions.

# 6 Ripen and Saturation

## 6.1 Local closure under staged inputs

Given a rule environment $\mathcal{R}$, a local entry $I$, and an injection plan $J$, ripen replays the observed preconditions, injections, and actions, then runs the local rules until a complete, uninterrupted sweep makes no update or an observation limit stops further admission. The statuses are deliberately distinct:

- `Running`: a registered local task is being advanced;
- `WaitingEquivalent`: a task awaits an already identified equivalent entry;
- `ObservedUnfinished`: a retained prefix stopped by an observation limit;
- `Rejected`: entry or validation obligations were not met;
- *Saturated*: an operational fixed point under the specified local model.

A local saturation result neither saturates all of tier0 nor certifies every embedding of the interface. `RipenInput(k)` is an opaque symbolic boundary node: a repeated index preserves identity, while different indices are initially unmerged. It is not a wildcard that matches every value. It does not recover the internal structure of an external eclass. Matches requiring that structure must be treated as additional coarse interaction.

If a second C requires a later external fact, the contract has the form

$$I_1; C_1; S_1; \mathrm{Inject}(I_2); C_2; S_2. \tag{8}$$

Injection declares a caller obligation; it does not derive the fact internally. Promoting $I_2$ into the initial entry without justification would change the activation conditions.

**Definition 2** (Saturated rule composition)**.** *A saturated rule composition consists of a validated composition, its trigger and input mapping, its staged obligations, a saturated realization K, and its certificate. In the current implementation K is a finite materialized body. Cataloguing equal K states does not identify two compositions with different triggers or obligations, and no live matcher reuse is performed. A symbolically saturated infinite body needs the stronger query contract discussed later.*

## 6.2 Sharing contents without conflating triggers

Different combinations may export identical local states while retaining different triggers, bindings, and injection plans; the current catalog records this equality but does not install shared bodies into tier0. The prototype compares complete exported typed graphs, equality classes, fixed ports, and row visibility. The

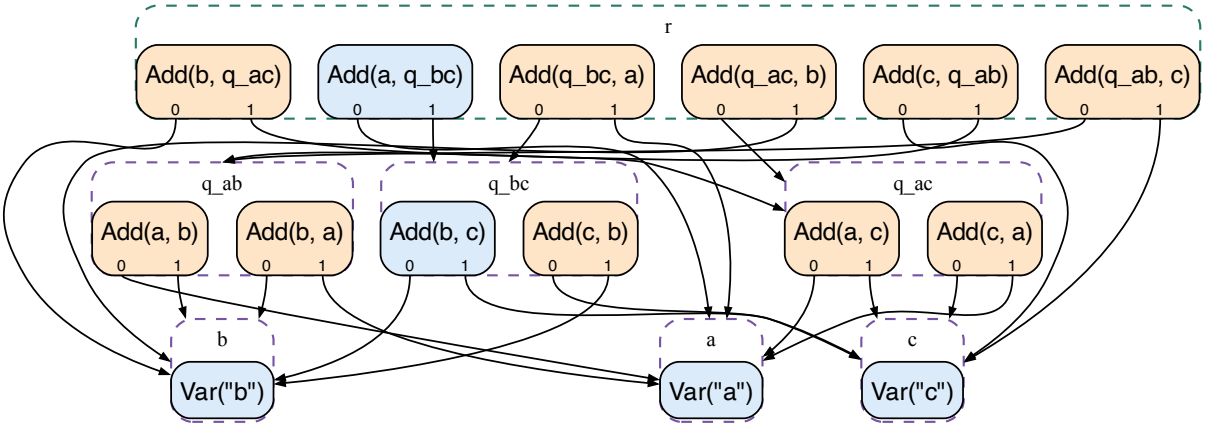



Figure 11: The Add body exported by the mini `math.egg` fragment. This figure is generated from egglog's own `-to-dot` serializer: dashed clusters are e-classes, solid boxes are e-nodes, and numbered ports preserve ordered child positions. The paper-side transformation only inlines Var string payloads and annotates the entry-relative colours; it checks that every Math node and ordered child edge from the native DOT survives. The graph has 12 Add nodes, three Var nodes, and seven Math e-classes.

rule environment must match. When updates or visibility matter, registration order and ruleset scheduling information are also retained.

**Proposition 2** (Composition of monotone closures)**.** *If* Cl *is extensive, monotone, and idempotent over a fixed universe, then*

$$\mathrm{Cl}(\mathrm{Cl}(A) \cup \mathrm{Cl}(B)) = \mathrm{Cl}(A \cup B). \tag{9}$$

*Proof.* Extensiveness gives $A \cup B \subseteq \mathrm{Cl}(A) \cup \mathrm{Cl}(B)$, yielding one inclusion by monotonicity. Conversely, both $\mathrm{Cl}(A)$ and $\mathrm{Cl}(B)$ are contained in $\mathrm{Cl}(A \cup B)$. Monotonicity and idempotence give the reverse inclusion. □

This identity motivates a possible future algorithm starting from existing closures and processing new interaction. The current pipeline instead reconstructs and replays members; it does not implement this identity as a body-composition algorithm. Their union need not already be saturated. The identity supplies no execution budget and is not a theorem about every egglog operation. In particular, subsume changes visibility, and a min/max merge alone does not prove that every surrounding query is monotone.

Exported-content equality is also not equivalence of arbitrary suspended incremental engines: the key does not serialize all of $\delta$. Direct reuse of native matcher or join state is not implemented. The current guarantee is sharing of declared local contents with separate trigger contracts.

### 6.3 Versioned evidence and recursive promotion

Historical saturated bodies are immutable; applicability of a concrete instance requires separately versioned evidence. The contract component models token, table, and scope subscriptions, union aliases, and invalidation by updates or subsumption. Unknown ownership requires conservative invalidation. These contract regressions should not be read as proof that the default native pipeline routes every operation through a subscription index: its CS registry tracks newly observed applications and interns explicit member sets.

A stale descriptor cannot authorize substitution without validation. A lease whose version has not changed is not by itself a proof of a current whole-graph embedding. Descriptor promotion was tested in earlier prototypes; automatic reuse of saturated bodies as tier0 execution units remains a separate step.




Same retained equality does not imply the same matching visibility.

Figure 12: The constant-fold and prune rules of `math.egg`. Native execution retains the Add row after subsumption but marks it hidden. An exported local state must record this flag; an instance lease depending on its visibility needs rechecking.

# 7 Two Histories and Reusable Origin Templates

## 7.1 Acquisition, interpretation, and reuse are different operations

Using egglog as both the tier0 engine and the isolated ripen engine does not require constructing the same history twice. The earlier runtime explicitly reused its full tracing and layer-building path inside every ripen cell. This produced a second application DAG with bindings, row producers, and local coarse/smooth layers. In the current path, saturation comparison and weak-dominate queries consume native states and committed-effect information; they do not consult that second layer history.

The distinction is operational. Tier0 still records versioned dependencies to construct CS interfaces. Ripen normally records committed writes and unions and summarizes inserted operators and observed changes, without retaining LHS binding lists or constructing local layers. Full local history remains an explicit option. This is selective acquisition and interpretation of trace data, not a claim that a logical match is a committed effect or that all tracing is disabled.

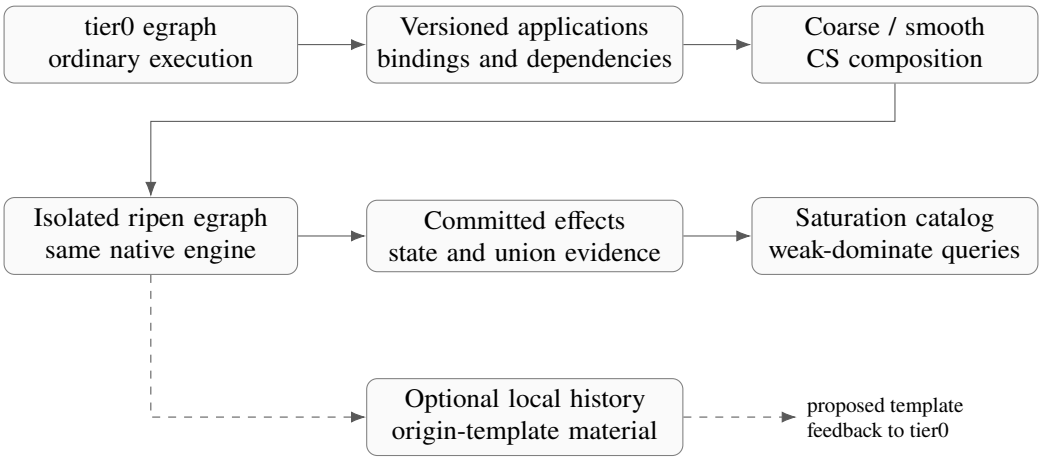



Figure 13: Two uses of tracing. Tier0 retains application provenance for composition analysis. Local ripen normally retains committed-effect summaries; full local binding/layer history is optional. Turning off that diagnostic history does not turn off tier0 provenance. Template feedback is a proposed consumer of the optional history, not a feature of the current execution path.

## 7.2 A finite origin-template contract (proposed)

The unused diagnostic history has a potential algorithmic role. Instead of rebuilding it for every instance, a future Bake step could compile one checked local derivation into

$$T = (P, S, D, \rho, \Gamma, \mathcal{E}). \tag{10}$$

Here $P$ is the typed caller interface, $S$ contains fact slots, $D$ is a finite application dependency DAG, $\rho$ routes rule inputs through slots and ports, $\Gamma$ retains alias, equality, guard, and visibility obligations, and $\mathcal{E}$ describes effects. An instance has the form

$$\text{Instance}(T, \theta, \Phi, O), \qquad \Phi : S \longrightarrow \text{typed actual facts and versions}, \tag{11}$$

where $\theta$ binds caller ports and $O$ records residual external obligations. Slots are logical identities; they need not denote distinct physical e-nodes when the interface permits aliasing or hashcons finds an existing row.

For example, use the actual association and commutation bodies from `math.egg`:

$$R_a : \ \text{Add}(x, \text{Add}(y, z)) \longrightarrow \text{Add}(\text{Add}(x, y), z), \tag{12}$$

$$R_c : \ \text{Add}(u, v) \longrightarrow \text{Add}(v, u). \tag{13}$$

After $R_a$, a particular application of $R_c$ reads the internal slot $s_1 = \text{Add}(x, y)$ with $u \mapsto x, v \mapsto y$. The template stores this routing once. In a checked instance, locating its input means resolving $\Phi(s_1)$ rather than rediscovering the dependency wiring.

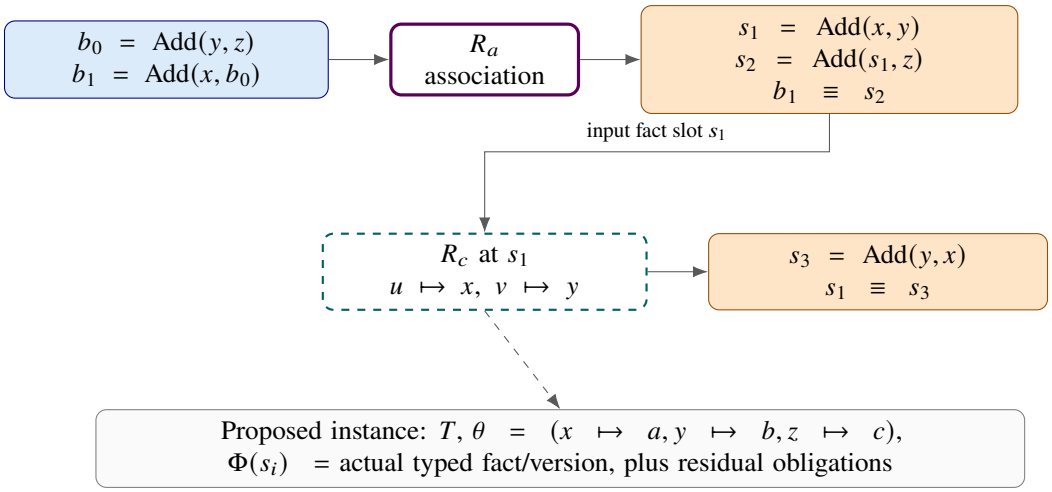



Figure 14: A proposed origin template using association and commutation from `math.egg`. Blue/orange boxes are LHS/RHS fact slots; purple/teal boxes are context-relative coarse/smooth applications. The second application reads a known internal slot instead of rediscovering its wiring. A slot can map to an already existing fact: this drawing describes a reusable derivation interface, not a claim that each orange slot caused a fresh tier0 insertion.

### 7.3 Derivational provenance is not execution provenance

The local and tier0 executions share rules and may share a structural derivation, but they need not have identical histories. A row inserted by local ripen may already exist in tier0. An actual write may be deduplicated; union may change canonical representatives or enable a previously unavailable match; subsumption or function updates may invalidate a read. The instantiated template can justify an internal derivation without becoming the recorded producer of a fresh row. An implementation must retain actual commit outcomes separately.

Coarse/smooth classification therefore remains relative to the instance. A step is smooth only if all its required facts and equality/visibility conditions are supplied by the selected interface and internal slots. Additional inputs remain coarse obligations. A template does not assign a permanent coarse/smooth label to a rule, nor does membership in the same eclass identify a unique producer. Reuse requires an explicit slot map, compatible rules and guards, and valid version/equality evidence at the current use site.

**Proposition 3** (Conditional reuse of a finite origin template)**.** *In the positive facts-and-equality fragment, let $T$ be a finite validated application DAG. If an instance satisfies its typed entry and guard obligations, and each input slot refers to an entry fact or an earlier established output, then topological replay supplies a valid derivational explanation for its outputs. This does not identify the actual tier0 producer of a deduplicated fact.*

*Proof.* Entry slots satisfy the base case. In topological order, every step receives its required facts from the entry or the induction hypothesis; its guards are assumed checked under the instance contract. Its trusted rule therefore establishes the mapped output facts or equalities. Deduplication preserves this logical result without implying a fresh insertion. Updates and subsumption require the stronger per-step visibility/version contract and are outside this positive-fragment claim. □

The implemented tier0 path already indexes producers; it does not repeatedly search the whole graph for each origin. The proposed gain is reuse of dependency metadata, relative bindings, and classification. It does not remove native Join work unless a later execution mechanism actually invokes the template under a validated interface contract. Nor is materializing $n$ output identities an $O(1)$ operation. Shared template metadata must still be charged together with instance bindings, slot maps, and residual facts. In particular, the current system has no automatic origin-template feedback to tier0.

# 8 Parameterized Rule Compositions

The following sections state summary contracts and retain scoped examples from earlier symbolic prototypes. They do not describe an automatic recurrence compiler in the current default native pipeline.

## 8.1 Stable binding and effect transfers

Repeated rule names are weak evidence. Two ABAB sequences can have different bindings; conversely, independently scheduled branches can represent the same dependency structure despite different event orders. We model a repeated step as

$$z_{k+1} = T(z_k), \qquad z_k = (\beta_k, \mathcal{E}_k, \mathcal{P}_k), \tag{14}$$

where $\beta_k$ is the binding, $\mathcal{E}_k$ accumulated effects, and $\mathcal{P}_k$ the frontier of positions and obligations. In an operational fragment, the effect component must retain structured updates and visibility, not merely a set of inserted facts.

A fractal rule composition is a parameterized family with a trigger, guard, stable transfer, effect contract, and observable interface. "Fractal" denotes recursive structural regularity here, not a claimed geometric dimension. A useful abstract form is

$$\mathcal{F} = (\mathit{Trigger}, D, T, \mathit{Guard}, \mathit{Effects}, \mathit{Queries}). \tag{15}$$

Finite histories propose such an object; they do not prove that it remains valid for arbitrary counts. Earlier transfer prototypes recorded Carry, Return, and External port origins and finite witnesses. General induction remains a separate proof obligation.

## 8.2 Trigger states and nonuniform prefixes

A stable phase may begin only after several injections and effect-producing steps:

$$\mathrm{Trigger}(I, \beta_0); T^k; \mathrm{Exit}\,. \tag{16}$$




Figure 15: The binding transfer of the integration-by-parts rule in `math.egg`. Growing expression depth does not require a new transfer template: the same Diff/Integral constructors and carried integration variable define each step.

Different bindings may require different triggers. An initial "dirty" prefix must not be omitted merely because the later transfer is regular.

For example, swapping Add arguments twice is an identity on argument positions, but the first swap may insert a node and perform a union. For a fixed position and fixed arguments, once both orders and their equivalence are present, further swaps add no local effect. The stabilization point depends on the initial graph; there is no universal rule equating two swaps with an empty execution.

Nor does a change of rule name in the log establish a turning point. Semi-naive scheduling and other work can interrupt a still-enabled family. A turning point must be defined by a guard, resource, interface, or effect change. Fractal operators need not commute or be orthogonal.

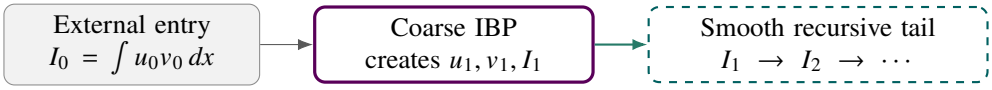



Figure 16: A one-step trigger for the selected IBP family. The first application acquires an external entry; subsequent residual applications reuse constructed facts. Other families can have longer, nonuniform effect-producing prefixes.

### 8.3 Multiple application positions

For a branching family, use an indexed position domain rather than flattening all steps into a sequence. A schematic binary generator can use

$$D_{k+1} = \{2i, 2i+1 \mid i \in D_k\}. \tag{17}$$

The text $A \to B + B$ alone does not prove physical doubling: identical B nodes can be hashconsed. Exponential e-graph growth additionally requires distinct bindings or positions that are not absorbed by existing equality. Three-way branching generally suggests $3^k$, not cubic growth in $k$.

We must therefore distinguish unfolded tree size, parameterized description size, event count, and deduplicated e-node count. A compact binding generator can represent a large family even when the resulting graph does not compress by the same factor.

## 9 Dissipative and Infi Rule Compositions

### 9.1 Dissipation is a property of obligations

**Definition 3** (Dissipative Rule Composition)**.** *Under a fixed external-supply contract, a fractal process is dissipative if a well-founded measure strictly decreases on every valid recursive step. A simple case is* $\mu(z_k) \in \mathbb{N}$ *with* $\mu(z_{k+1}) < \mu(z_k)$.

The measure applies to progress steps of the represented process; redundant re-execution of an already accounted match is not a new recursive step. The decreasing quantity may count remaining positions, an

unprocessed index range, or outstanding obligations. It does not necessarily count facts being deleted from the e-graph. Facts are generally reusable, so a need for external facts does not itself imply dissipation. Replenishing the external supply may invalidate a phase-local ranking argument.

For a finite sum,

$$(i, s) \mapsto (i + 1, s + x_i), \qquad 0 \le i < n, \tag{18}$$

$\mu = n - i$ proves termination. In reducing five ones to five, it is the remaining reduction positions that decrease. The constant node one can remain a single shared node throughout, and need not disappear at all.

A branching process can also be dissipative while its levels become wider. Suppose each task of rank $r > 0$ creates at most $d$ tasks of rank at most $r - 1$, and rank-zero tasks terminate. On a finite frontier, use

$$\mu = \sum_{p \in \mathcal{P}} (d + 1)^{r(p)}. \tag{19}$$

Replacing a positive-rank task adds at most $d(d + 1)^{r-1}$, less than the removed weight $(d + 1)^r$. Thus bounded-depth binary expansion can have exponentially wide levels and still terminate. The removed objects in this argument are pending tasks, not retained e-graph facts.

Repeated integration by parts offers another candidate: polynomial degree may decrease if the correct factor and variable are repeatedly differentiated. This needs a polynomial and differentiation contract; finite examples do not prove that arbitrary integral rewrites terminate. We do not report such a general proof as an implemented capability.

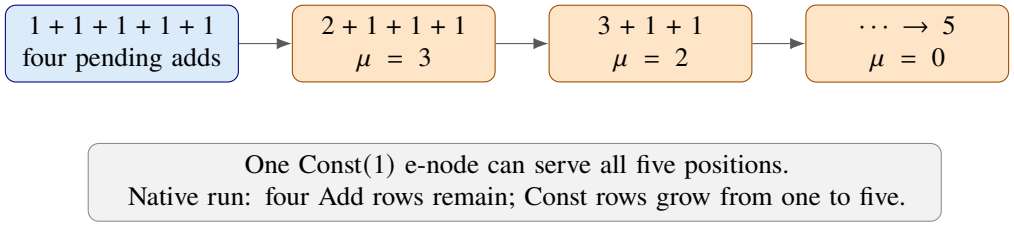



Figure 17: A dissipative composition using the native constant-fold rule from `math.egg`. The initial expression is left-associated; outer Add facts are part of the declared entry. The remaining work frontier decreases, not the stored fact set. No pruning is enabled in this example.

## 9.2 Infi rule composition as an unbounded prefix interface

**Definition 4** (Infinite-prefix family)**.** *For a fixed admitted trigger instance, an infi rule composition admits, under its invariant, a valid prefix of length $k$ for every natural number $k$. Claiming unbounded growth of the physical graph further requires a freshness argument for facts or positions.*

The initial instance is fixed: choosing a larger terminating input separately for each $k$ does not establish this property. This definition concerns a family of finite prefixes. With nondeterministic choices, arbitrarily long paths do not by themselves establish that one selected execution continues forever. A fixed transfer and consistent prefixes provide a stronger contract.

We distinguish stabilization of effects, bounded periodic representations, and genuinely unbounded generation. Pure smooth execution does not imply the last case, and budget exhaustion proves none of them. Unknown remains an essential outcome.

For example, mathematical-natural-number payloads can describe $A(k, i) \to B(k, 2i), B(k, 2i + 1)$ and $B(k, i) \to A(k + 1, i)$. Machine i64 arithmetic additionally requires overflow guards or bounded depth; it cannot inherit an unbounded mathematical conclusion without qualification.

### 9.3 A productive example from math.egg

Let $D_x u$ denote `Diff(x,u)`, let $A_x v$ denote `Integral(v,x)`, and define $I_k = \text{Integral}(u_k v_k, x)$. The actual integration-by-parts rewrite is:

```
(rewrite (Integral (Mul a b) x)
  (Sub (Mul a (Integral b x))
       (Integral (Mul (Diff x a) (Integral b x)) x)))
```

It gives the uniform transfer

$$u_{k+1} = D_x u_k, \qquad v_{k+1} = A_x v_k, \qquad I_k \Rightarrow u_k v_{k+1} - I_{k+1}. \tag{20}$$

In the IBP-only fragment, with distinct symbolic Var leaves and no additional equalities, the derivative and antiderivative arguments grow structurally. The residual is again an eligible Integral-of-Mul node. The two constructors are never simplified by another rule, so this supplies arbitrarily long finite prefixes with fresh nodes. No numeric loop counter is evaluated in the graph. In this fragment, each step creates

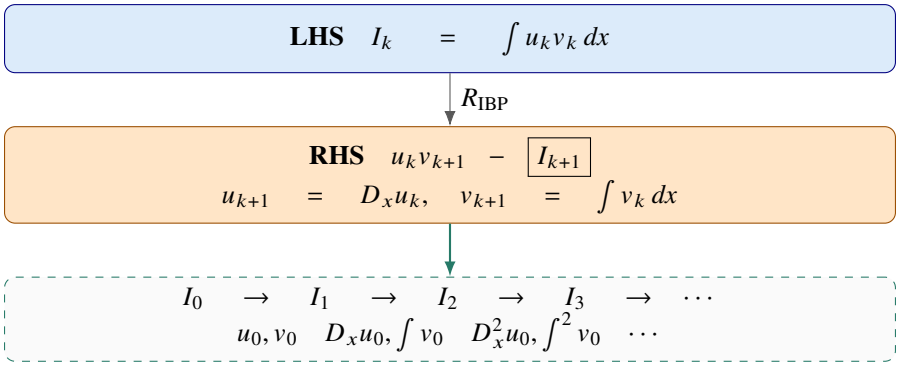



Figure 18: A productive infi rule composition from the actual IBP rule. In the *IBP-only* fragment, the boxed residual regenerates the same pattern with strictly deeper Diff/Integral arguments. Four native steps yield 11, 17, 23, and 29 Math e-nodes. The all-depth argument requires the restricted theory; full `math.egg` may simplify the derivative and terminate this family.

one Diff row, one antiderivative Integral, one residual Integral, two Mul rows, and one Sub row. Distinct derivative and antiderivative arguments prevent their keys from coalescing; IBP unions the integral result with its Sub representative, not with either argument. Thus the constructor-row count is $N_{\text{Math}}(k) = 5 + 6k$, excluding primitive String values and implementation-only global bindings. A native check of prefixes $k = 0, 1, 2, 3, 4$ gives 5, 11, 17, 23, 29 Math e-nodes. These finite measurements check the illustration; the all-depth argument depends on the restricted theory. In full `math.egg`, derivative simplification, other integration rules, and pruning can stop or change the recurrence. With a polynomial derivative contract, the same IBP pattern can instead become dissipative.

### 9.4 A recursive DSL that becomes an array contract

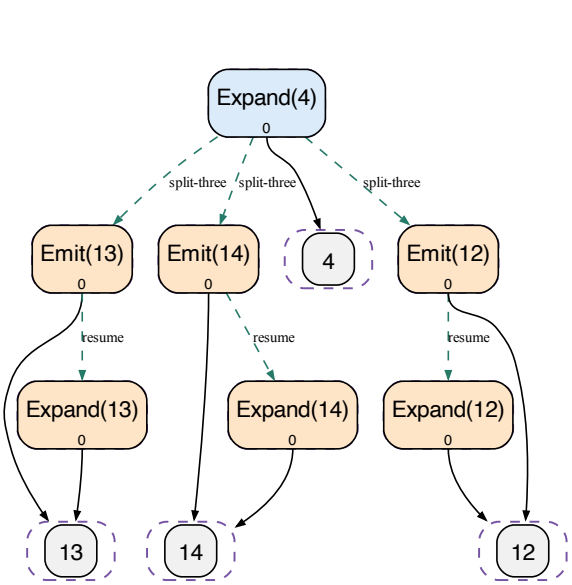
The previous examples use rules copied from `math.egg`. The following four small programs are deliberately written as a new recursive DSL. They use only a cursor, an upper bound, and a ternary expansion; no array constructor is present in the source. Bake observes the native traces, extracts the repeated binding/effect transfer, and then applies its two structural array-law checks. The generated library is therefore a useful test of the abstraction boundary: the array appears after recognition, not as a hand-written replacement rule.

Listing 1: The two source families supplied to Bake

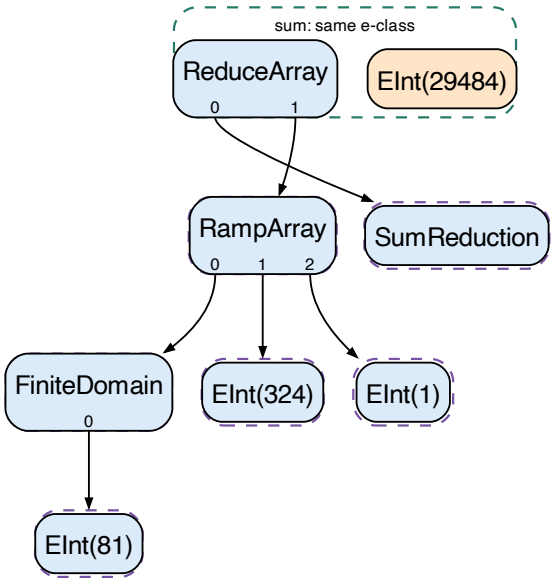
```
; scan-short.egg and scan-long.egg
(datatype Math (Cursor i64 i64) ...)
```

| Observed family | Recognized law | Array query produced by Bake |
|---|---|---|
| Cursor$(i, end)$ | unit stride; measure $end - i$ decreases | RampArray$(end - start)$; start = 3, limit = 11 gives values $4, \ldots, 11$ and sum 60 |
| Expand$(n)$ and Emit$(n)$ | radix frontier; residues $0, 1, 2$ cover the next level | depth 4, start = 4 gives $3^4 = 81$ values $324, \ldots, 404$ and sum 29484 |

Table 1: A complete path from a recursive rule DSL to a first-class symbolic array query. Both Bake queries report zero tier0 rule applications, zero element queries, and zero expanded prefixes. The query is over the certified value projection, not over all intermediate facts, unions, or effects.




Native recursive e-graph: `Expand(4)` produces `Emit(12)`, `Emit(13)`, `Emit(14)`.




Native array-query graph after Bake: `RampArray(81,324,1)` and `ReduceArray`.

Figure 19: The recursive-array case in two stages. The left graph is an executable Math e-graph snapshot; solid edges are native child references and dashed `split-three`/`resume` edges are explicitly labelled rule-instance annotations. The right graph is the native tier2 array query produced from the certified Bake law. Its paper annotation removes helper arithmetic alternatives, but the remaining nodes and solid child-class edges are checked against the native DOT export.

```
(rule ((= node (Cursor i end)) (< i end))
      ((Cursor (+ i 1) end)) :name "scan-next")

; tree-one.egg and tree-four.egg
(datatype Math (Expand i64) (Emit i64))
(rule ((= node (Expand n)) (> n 0))
      ((Emit (+ (* n 3) 0))
       (Emit (+ (* n 3) 1))
       (Emit (+ (* n 3) 2))) :name "split-three")
(rule ((= node (Emit n))) ((Expand n)) :name "resume")
```

The archived manifest contains two samples for each family. An earlier Bake run on `examples/recursive-array/manifest.json` produced two portable fractal templates (this is not a command of the current native CLI): `fractal_0000` is a `linear_extension` with a `bounded_unit_stride` law; `fractal_0001` is a `recursive_dag` with a `radix_frontier` law of radix three. The identifiers are local to this library; the law kinds and contracts are the meaningful results.

The first family is dissipative because the remaining distance to the limit strictly decreases. The second family has finite prefixes whose frontier grows by a factor of three; its finite array query is valid only when

the requested frontier fits the checked i64 domain. The code checks the guard, constructor shape, residue coverage, return rule, and source schema before emitting either law. It does not infer an array law from matching counts alone.

# 10 Feedback from Fractal Compositions to Ripen

## 10.1 Finite scheduling and reduction

A proved dissipative family supplies a principled local count or budget. Ripen can execute its trigger, finite repeat, and exit as a unit, rechecking the contract when external supply or binding changes. A ranking for one family does not prove that all of ripen terminates: another family may reactivate it.

The existing tier2 DSL includes an explicit Reduce node and exact endpoint laws. For a natural count $k$ and a fixed per-step contribution $c$,

$$\mathrm{Reduce}(\mathrm{AddConstant}(c), k, t) = t + kc. \tag{21}$$

For contributions $a + id$, $i = 0, \ldots, k-1$, the accumulated contribution is

$$ka + \frac{k(k-1)}{2}d. \tag{22}$$

These are laws for a supplied accumulation contract, not an algorithm discovering that contract from arbitrary history. They concern exact algebra; floating-point reassociation and machine overflow require additional justification.

## 10.2 A worked endpoint and intermediate-fact example

Consider the guarded recurrence

$$R: \ f(t) \Rightarrow f(t-1) + 1, \qquad t > 0. \tag{23}$$

For natural $n$ and $0 \le k \le n$, induction yields the endpoint $f(n-k) + k$. Setting $k = n - m$ gives

$$f(n) \Rightarrow f(m) + n - m, \qquad 0 \le m \le n. \tag{24}$$

The guard provides a dissipative measure. Removing it over mathematical integers instead permits an unbounded descending sequence, subject to a freshness and effect argument; the same machine-integer implementation is still bounded by its numeric semantics.

The endpoint does not contain every intermediate fact. Another rule may need $f(n-j)$ at a particular $j$. A reusable summary therefore needs a query interface for $0 \le j \le k$, together with the corresponding equality/effect information. Replacing the entire history with only the last expression can suppress such interactions and is not a sound implementation of general ripen.

For the IBP example, the actual unrolling is nested Sub nodes. Turning that nesting into the alternating-sum endpoint requires additional exact additive laws. The IBP-only native trace is not claimed to perform this normalization.

## 10.3 Arrays and symbolic coverage

A useful summary contract exposes a trigger, iteration domain, binding transfer, effects, endpoint queries, and intermediate-position queries. Finite domains naturally give fold and array abstractions; infinite-prefix families require lazy instantiation. Existing logical ArrayGet, Tabulate, Map, and Fold constructs provide relevant vocabulary, but do not by themselves implement mutable memory, alias analysis, cache placement, or GPU scheduling.

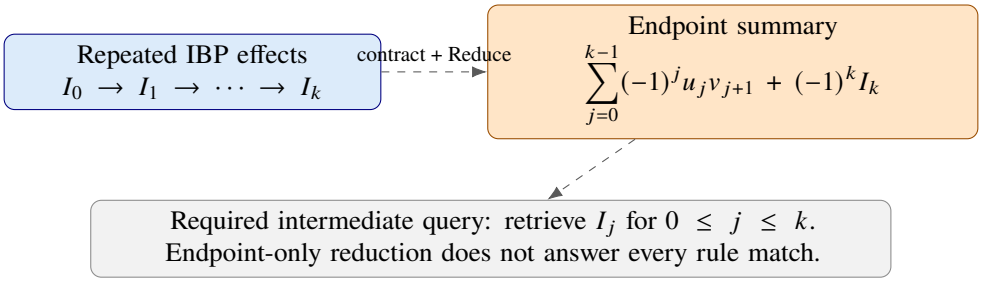



Figure 20: How an infi composition could feed ripen: summarize the alternating endpoint, but retain an indexed residual interface. This is a proposed contract for the worked IBP family with additional exact additive laws, not a claim that the current Reduce DSL automatically discovers or executes this alternating-sum summary.

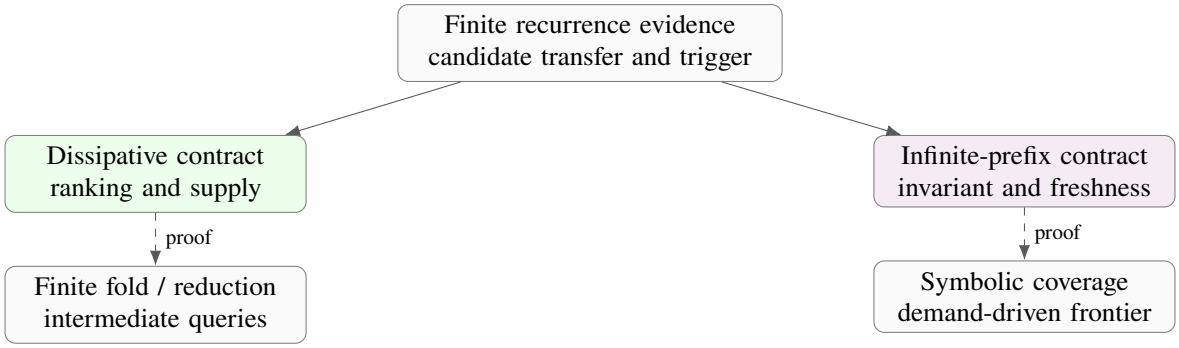



Figure 21: Proof obligations separate observed regularity from summary-based execution. A finite witness is not a certificate of either termination or unbounded generation.

An infi rule composition that continually generates fresh facts generally has no finite materialized saturated body. We propose a distinct `SymbolicallyCovered` status: within a specified observation interface, internal matches and effects are represented by a summary and required positions can be produced on demand. This needs both sound answers and coverage of relevant internal matches. If a query cannot be answered, ripen must retain a frontier, expand the requested position, or return unknown. It must not label budget-limited sampling as closure.

### 10.4 Bake and the present integration boundary

Bake can aggregate observed families from multiple examples into a frozen library. A new example matches the library under explicit binding/effect contracts rather than silently assuming that training examples proved generality. Earlier local prototypes included narrow, name-independent proof schemas for a guarded unit-stride scalar recurrence and a bounded radix-frontier payload query [2]. The former preserves the limit and other fields; the latter checks residue coverage and machine-range bounds. Their certified queries do not materialize all tier0 effects.

These narrow queries are useful evidence that executable summaries are possible. They are not a complete fractal rule composition-to-ripen integration. The current native ripen path does not execute such summaries. General induction and a complete internal-match interface remain open.

Listing 2: Target feedback protocol. The certified-summary branches are not yet generally integrated.

```
on committed event e:
    notify subscribers of changed resources
    update witnessed CS interfaces
    register observed CSCS and guarded CCSS interfaces
```

```
for each registered interface U:
    validate staged external obligations and bindings
    if a verified dissipative summary applies:
        guide ripen using its finite domain and effect contract
    else if a verified infinite summary covers required queries:
        retain SymbolicallyCovered and a demand-driven frontier
    else:
        run ordinary bounded ripen
    catalogue equal exported states, retaining trigger obligations
    promote certified results and notify subscribers
```

# 11 Implementation and Evidence

## 11.1 Current implementation scope

The runtime account in this revision uses the September 23 changes `3bd9c8d`, `7dcd5dc`, and `df8b122` [4]. Earlier experiment records below retain their original scope and revision.

| Component | Current scope |
|---|---|
| Tier0 history | Committed-event evidence, relative bindings, and Rust CS layers. |
| CS registration | Observed consumer connections and exact member interning; no default all-pairs search or historical Use admission gate. |
| Ripen | Staged validation followed by isolated native execution; optional local layer history, committed-effect observation by default. |
| Saturation equality | Typed exported-state bijection with fixed ports and visibility; refinement followed by exact search without a search cutoff. |
| Weak dominate | Native typed Join queries; exact existence and fresh-match witnesses, or optional exhaustive binding lists. |
| Growth/reentry | Retained unfinished states, component buckets, and observed trigger comparisons; not a general loop-body induction procedure. |
| Origin-template feedback | Contract and example in Section 7; not connected to tier0 execution. |

Table 2: Current mechanisms, distinguished from the historical prototypes and proposed runtime reuse.

The old limits on CS versions, members, proposal counts, and candidate partners are not defaults of this path. Registered observed interfaces are processed without a Top-$K$ admission policy. This does not make discovery complete: it still follows observed consumer connections, and unsupported interface or replay contracts can be rejected. Resource limits on each ripen execution are separate from candidate selection.

## 11.2 Native observation limits and safe interruption

The CLI and local web service share defaults of 50 ms of cumulative advancement wall time and 200,000 facts per cell, with four passes per sweep. These govern advancement after preparation, not the entire program or process RSS. The engine checks at native Join and match-admission boundaries before accumulating an unbounded new batch of trace records. Instruction-based pending-write reservations are conservative for the supported action path: deduplication may make the committed row increase smaller, so admission can stop below 200,000 actual facts. The report distinguishes `pending_fact_reservations` from a measured fact-count threshold.

Already admitted actions commit and required rebuild completes. A long primitive, index operation, or rebuild can exceed the nominal deadline; this is not a hard real-time or memory guarantee. An interrupted query does not establish a fixed point even if it reports no committed update. Its seminaive cursor must not skip unvisited inputs if execution is resumed. Entry preparation, exports, queries, and cleanup contribute additional end-to-end cost.

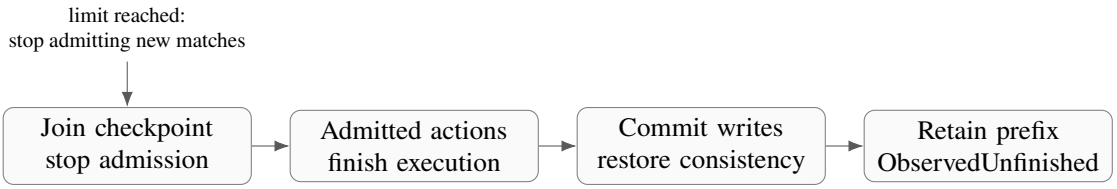



Figure 22: The stop path is cooperative. A time/space observation stops new work, not already admitted mutations or their required rebuild. The retained prefix is not a saturated body. Success of an existential query uses a separate read-only stop signal; it does not certify saturation either.

### 11.3 Exact predicates instead of exhaustive diagnostic bindings

Let $M_q(G)$ be the binding set of a typed query in the current graph. For a fixed cell, the two relevant predicates are

$$M_q(G) \neq \emptyset, \qquad M_q(G) \setminus \mathrm{canon}_G(M_q(G_{\mathrm{initial}})) \neq \emptyset. \tag{25}$$

The canonicalization is necessary after union. A fresh binding proves both predicates; a read-only Join can then stop cooperatively. If no fresh binding is found, it must exhaust the Join to establish the negative result. This is exact existential evaluation, not sampling a fixed number of representatives. The returned binding list is marked non-exhaustive; complete enumeration remains available for debugging.

Query keys preserve types, literals, repeated-variable aliases, and projections, while normalizing variable numbering and omitting unused boundary vertices. Compiled queries and unchanged-state results are reused within their compiling engine. Native updates invalidate the result cache. An empty required initial table proves the initial Join empty without compiling it. Probes use a flat Generic Join plan to avoid repeated tree-decomposition/materialization setup; ordinary rewrite planning is unchanged. Exact whole-state comparison remains a separate operation, not an existential embedding query.

### 11.4 Isolation and separating execution from presentation

Independent cells can be prepared, advanced, queried, and released in parallel in the graph-free path. Results are published in a fixed order. A correctness bug found during this work was that database clones shared their pending-table notification queue: resetting one clone could consume another clone's pending notifications. The database clone now copies that queue into an independent domain, while workers of the same database still share its notifications. Compiled query collectors remain engine-bound and are not concurrently reused across unrelated clones.

The implementation also reuses prepared ASTs, maintains event/producer indexes incrementally, uses typed native values when indexing exported vertices, and serializes local reports directly from borrowed structures. Disabling DOT/SVG skips their generation; graph-free CLI runs do not construct per-round display snapshots. When enabled, SVG layout is a separate bounded worker phase after analysis. Its wall time must not be attributed to native saturation.

### 11.5 Three-round math experiment and regression boundary

The native experiment uses `experiments/math_mini3/source.egg`, not the full eleven-round workload. It keeps the 50 ms / 200K defaults and four analysis workers, disables DOT/SVG, and times the compiled process without sampling or compilation. Table 3 reports all three recorded runs, including the first launch [4].

The run contains 79 saturated jobs (not 79 distinct catalog bodies) and six unfinished jobs. The 79 completed state structures agree with the preceding baseline after excluding run-local IDs, and 488 weak-dominate decisions whose source is completed agree. This does not assert equality of time-limited unfinished prefixes: faster code can advance them farther before the same deadline. A native execution-only

| Run | Process wall time (s) | Peak RSS (MiB) |
|---|---|---|
| First recorded launch | 1.33 | 624.0 |
| Second | 1.06 | 595.8 |
| Third | 1.07 | 585.0 |

Table 3: Graph-free native analysis, release ThinLTO. These few measurements are not confidence intervals, a real-time bound, or a claim of a consistently sub-second runtime.

measurement has an internal timer near 5 ms; that timer must not be divided into the complete analysis process time as if the measurements had the same boundary. The present work reduces analysis overhead, not tier0 execution work, and the requested sub-second end-to-end target is not consistently met.

DABCABC and branching regressions use a common fact-admission limit and a large time allowance so the stopping reason is not machine speed. Full local history and effect-only execution produce the same tested state/evidence observations. The tests require several repeated A/B/C applications, keep D as a one-time prefix, and forbid labelling the unfinished body saturated. They explicitly leave the minimal starting condition unresolved: passing these tests does not mean that the implementation has inferred ABC as a general recurrence. Other regressions compare existential queries with full enumeration through union and subsumption, check notification isolation, and reject reuse of rolled-back row producers.

### 11.6 Cost model and its limits

For $N$ imported applications, a persistent balanced event index costs $O(N \log N)$ over insertion, rather than reconstructing the whole index for every batch. With an unchanged scope, producer filtering visits the newly added certificates; a rollback can still require a full scan. If $B$ denotes total input/binding data and $Q$ the distinct normalized queries, these bounds do not remove the cost of processing $B$, compiling $Q$ queries, or executing their Joins. For $k$ evaluations of one query on an unchanged native state, the result cache avoids $k - 1$ repeated executions, but not the output/instance bookkeeping.

With independent tasks of costs $c_i$ and $p$ workers, parallel scheduling cannot beat either $\max_i c_i$ or $\sum_i c_i/p$; dependency, allocation, and publication costs remain. No end-to-end polynomial bound follows for saturation, state isomorphism, or an output-heavy Join. The observation limits are operational protection, not proofs of termination or a substitute for reporting unresolved work.

### 11.7 Historical recursive promotion and locality

The historical fixed recursive-CS experiment contains three source chains and is processed by bounded isolated ripen jobs. It reports nine closed jobs, seven distinct representative bodies, and six promotion records at maximum composition depth two. Its counters include six layer visits, 75 pair checks, 14 resource notifications, and three subscriber wakes [3]. A promotion adds a new composition descriptor to the bounded candidate store; it does not install a macro into tier0 or substitute a shared body during native matching.

This establishes that validated composition records can be fed back into the candidate store. It does not establish compression, acceleration, or reuse by the live egglog matcher. In a separate registry test, union of two resources wakes their two subscribers among 100 independent units. Shared constants, common tables, or unknown mutation ownership can legitimately require more notification work.

### 11.8 Historical records are not a controlled ablation

The first three records use the same named input and nominal attempt budget, but reflect different implementation versions. They document an evolution rather than isolate a causal improvement. In particular, explicit CS composition did not exceed the earlier cone record in shared-state count.

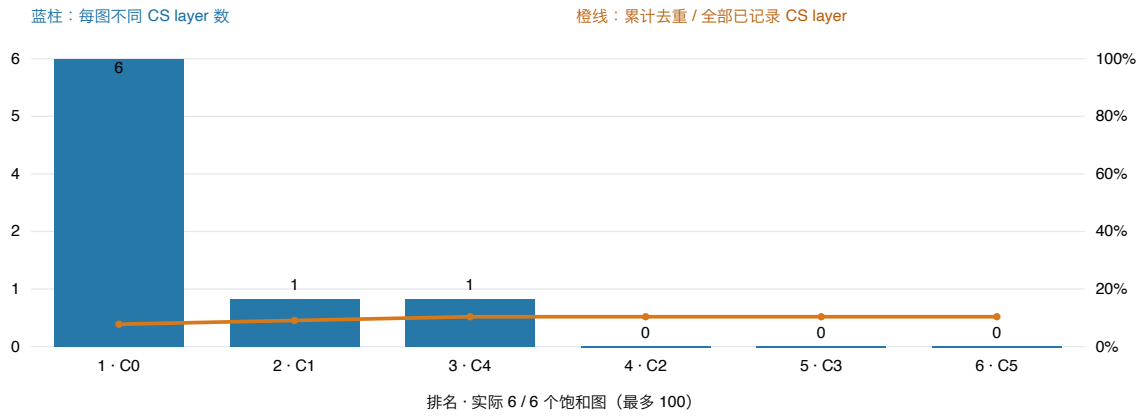



Figure 23: Historical ripen reuse coverage from the default math4 run introduced in commit `a4e148e`. Each blue bar is one SaturatedRuleComposition body, ranked by the number of distinct base CS layers whose complete member set reached exact saturation equality. The orange line is cumulative coverage after de-duplicating layer IDs across bodies. The run has six catalogued bodies, eight verified layers out of 77 recorded base CS layers (10.4%), six unattributed triggers, and queue states 15 Saturated, 1 Rejected, and 151 Pending. Pending layers remain unknown; this is a provenance-backed reuse lower bound, not a tier0 replacement rate or a storage-compression measurement.

| Recorded configuration | Saturation results | Shared states |
|---|---|---|
| microbench6 / Use only | 14 | 5 |
| microbench6 / ancestor cones | 18 | 7 |
| microbench6 / explicit CS | 16 | 6 |
| Three-CS recursive example | 9 | 7 |

Table 4: Historical prototype observations. Here microbench6 is `math-microbenchmark.egg`, not the full `math.egg`. These are not all rerun at one commit; the last row also uses a different input and budget.

A separate Python storage prototype expanded 100 C3 triggers into 1,200 Add occurrences, then globally deduplicated them to 304 rows. Retained object bytes including its index and union-find decreased from 25,740 to 20,540, about 20.2%, while queries and joins became slower. This was not egglog RSS and did not replace native tables. It illustrates why sharing potential must be measured together with access and maintenance costs.

For a single snapshot, if $N(U_j)$ is the e-node set covered by instance $j$, coverage must use a union:

$$\text{Coverage} = \frac{|\bigcup_j N(U_j)|}{|N(G)|}. \tag{26}$$

Multiplying instance counts by template sizes overcounts overlap. Net storage must also include dictionaries, bindings, residual facts, indexes, certificates, versions, and any retained raw history. Shared output identities must remain consistent across instances.

## 12 Limitations and Research Agenda

Finite recurrence evidence is not induction. Frequency and compression ratio do not replace guard preservation, a ranking argument, or query completeness. Likewise, recognizing a dissipative family does not prove that other rules cannot restart it.

Exported-state equality and live instance substitution remain separate. Opaque parameters omit external structure; version tracking prevents unchecked reuse but does not itself validate a current whole-graph

embedding. Subsumption and mutable tables require their operational contracts rather than the pure closure identity.

The discovery procedure is intentionally incomplete. It follows observed consumer connections rather than promising every useful subgraph. The absence of historical candidate-count cutoffs does not bound the number of observed compositions; per-cell ripen limits do not bound all discovery or query costs. Source-environment checks and report construction still have costs. Deeper recursive descriptions can also increase maintenance overhead.

The next evaluation should run all variants at one revision, with matched inputs and observation limits, separately ablating CSCS, CCSS, recursive promotion, subscriptions, and certified fractal summaries. It should measure repeated internal matching, retained memory, interface-query cost, invalidation/revalidation cost, and omitted candidates, including workloads on which the method loses.

This report uses locally available source and bibliography metadata only. It does not provide an exhaustive related-work survey or establish novelty priority over unreviewed work on graph grammars, library learning, or recursive summaries.

# 13 Conclusion

We describe an event-driven analysis pipeline for locality in recorded rule history. Coarse and smooth labels, CSCS/CCSS candidates, staged replay, and exact comparison of exported local states are implemented under explicit observation limits and contracts. The result is a catalog of conditional symbolic-interface observations; it is not a replacement execution engine and it does not make equal local states interchangeable inside tier0.

The current runtime additionally separates optional local provenance from committed-effect observation. Origin templates could reuse that provenance through caller bindings and fact slots, but the corresponding tier0 dispatch and validation mechanism remains unimplemented.

The recurrence and summary material defines proof obligations for future work. Earlier prototypes recorded finite transfer evidence and narrow checked query schemas; the current native path does not apply general fractal summaries. Proving arbitrary-depth induction, covering all relevant internal matches, and measuring a net runtime or storage benefit remain open.

# References


[1] Y. Zhang, Y. R. Wang, O. Flatt, D. Cao, P. Zucker, E. Rosenthal, Z. Tatlock, and M. Willsey. Better Together: Unifying Datalog and Equality Saturation. *Proceedings of the ACM on Programming Languages*, 7(PLDI), Article 125, 2023. DOI: 10.1145/3591239. Bibliographic metadata and abstract checked against local `egglog/CITATION.bib`; full text not independently reviewed for this report.

[2] *fractal_egraph local prototype*, commit `258eef4`, 2026. Core mechanisms: `src/native_cs.rs`, `src/closure_contract.rs`, `src/saturated_rule_composition.rs`; summary schemas: `src/native_bake.rs` and `rules/reduce.egg`. The accompanying verification manifest identifies additional source evidence. Implementation evidence, not an external publication.

[3] Local experiment records and regression tests, 2026. Records under `experiments/closed_storage/`: `closure-contracts-results.json`, `dependency-candidates-math6.json`, `cs-composition-math6.json`, and `results.json`. Reproduction commands appear in Appendix A; additional test references are listed in the verification manifest. Records are scoped observations, not a uniform benchmark.

[4] Native runtime and provenance update, 23 September 2026. Commits `3bd9c8d` (clone notification isolation), `7dcd5dc` (effect observation and exact existence probes), and `df8b122` (isolated parallel

analysis). `experiments/math_mini3/near_second.json`, `tests/reentry_history_regression.rs`, `tests/native_fact_query.rs`, and `tests/native_clone.rs`. These are implementation records, not external publications or a proof of general fractal discovery.

# A Reproduction and Evidence Levels

## A.1 Top-N ripen-body supplementary graphs

The reuse curve in Figure 23 ranks the bodies by the number of base CS layers that their exact catalog state covers. The companion exporter `tools/export_ripen_body_svgs.py` takes the same queue/catalog directory, selects the top *N* rows, and writes one SVG and DOT file per body.

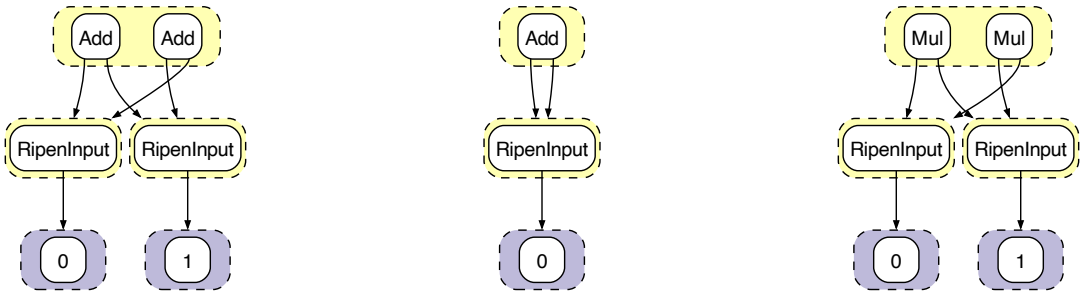
The exporter reads the canonical state file `catalog/states/state-NNNN.json` to select the top bodies, then copies `native-egraph.dot` and `native-egraph.svg` written by the actual ripen EGraph at its export point. If an older run predates native artifact export, it reruns that cell's recorded `entry.egg` in a temporary output directory and records `regenerated_native_artifact` in the manifest. No Node/Row carrier is reconstructed. These files are visualization evidence of the completed local run, not a tier0 substitution or a proof that the body can be injected into a live e-graph.

The checked default math4 supplement contains top bodies C0, C1, and C4. Its manifest records body sizes, layer coverage, source state files, generated DOT, and generated SVG. Reproduce it with:

```
python3 tools/export_ripen_body_svgs.py \\
  out/default-ripen-math4 out/ripen-body-top3 --top-n 3
```

The output directory is intentionally separate from the committed paper assets; copying it into a release artifact must preserve the manifest and state files.

Figure 24 embeds the three checked graphs. They are serialized directly from the three isolated egglog EGraphs after ripen reaches `Saturated`; the PDF conversion changes only the file format. In particular, these are Math-DSL graphs rather than a second graph whose generic `Node` and `Row` constructors encode the catalog JSON. C0 and C1 both arise from Add commutation, but C1 has an aliased boundary and therefore contains one `RipenInput` and one self-Add node. C4 is the analogous Mul body. The committed DOT files are the supplementary data from which the panels are rendered.




(a) C0: 6 covered layers (b) C1: 1 covered layer (c) C4: 1 covered layer

Figure 24: Top three exact saturated bodies in the default four-round math capture, ranked by distinct covered CS layers. Dashed rounded regions are e-classes; constructor boxes and ordered child ports are emitted by egglog's native EGraph serializer. C0 and C1 are Add bodies with different boundary aliasing; C4 is a Mul body. The corresponding `.dot` and `.svg` files are retained beside the paper figure.

## A.2 Native DOT figures

`examples/math-mini-trace.egg` marks three executable prefixes: entry, one associativity round, and saturation. The generator runs each prefix through native egglog with `-to-dot -to-json`. Raw DOT and JSON are retained under `figures/native/`. The transformation inlines primitive String leaves into Var labels, preserves every Math node and ordered child-class edge, and colours nodes by comparison with the entry. It validates the exported 2/4/12 Add counts and 5/6/7 Math-class counts, and writes these checks to `evidence.json`. No conceptual composition edge is inserted into the native e-graph.

```
make -C docs/papers native-egraph
make -C docs/papers pdf VERSION=v0.2.5
```

## A.3 Composition regressions

The paper uses the revised composition terminology. Legacy implementation identifiers remain unchanged: `CoarseRuleComposition` and `SmoothRuleComposition` label context-relative application forms; `SaturatedRuleComposition` stores one exported local state; it is not a trigger-bearing macro or a replacement for the tier0 e-graph. `Saturated` is the serialized status corresponding to a completed isolated fixed point; an interrupted prefix uses `ObservedUnfinished`. The colored examples are checked by `cargo test -release -test paper_math_examples`. Their complete scoped programs are in `docs/papers/examples/`.

```
# Native finite-property and reentry regressions
cargo test --offline --test core_flow --test reentry_history_regression \
  --test native_fact_query --test native_clone --test native_execution_limits \
  --test compact_trace --test table_support --test wildcard_roundtrip

# Complete analysis; no DOT or SVG. Keep compilation outside timing.
cargo build --release --offline --bin egg_layout
EGG_LAYOUT_GRAPHS=0 /usr/bin/time -l target/release/egg_layout analyze \
  --source experiments/math_mini3/source.egg --output out/paper-mini3

# Optional diagnostics (independent switches)
# EGG_LAYOUT_RIPEN_HISTORY=1: local application/layer history
# EGG_LAYOUT_FULL_TRACE=1: exhaustive weak-query bindings
```

| Evidence | Supported statement | Not implied |
|---|---|---|
| Committed event | A recorded effect occurred | Every binding can execute it |
| Finite witness | A transfer fits observed cases | Arbitrary-count stability |
| Local saturation | Fixed-entry local execution stabilized | Every embedding is saturated |
| Exact state match | Declared contents can be shared | Triggers are interchangeable |
| Ranking proof | Termination under its supply contract | Termination under all injections |
| Symbolic query contract | Coverage of specified observations | No observer needs intermediates |

Table 5: Evidence levels used throughout the report.